Faglige metoder i prog

DEBUGGING, FEJLFINDING!

1: SYNTAX-FEJL - fanges ved kompilleringstidspunktet eller af editoren undervejs!
(koden stemmer ikke overens med java-sprogets syntaks, og det fanges *før* kørslen)

2: RUN-TIME-FEJL - programmet chrasher på et tidspunkt under kørslen!
(man beder computeren om at gøre noget med programmet, den ikke kan under kørslen)

3: LOGISKE FEJL – programmet kan kompilleres, og det kører, men slet ikke som tiltænkt!
(programmet opfører sig forkert i forhold til ens idé, men det er korrekt syntaktisk og kører)

Ved run-time-fejl får man jo en tilbagemelding om fejltypen – fx:
NullPointerException, ArrayIndexOutOfBoundsException, NumberFormatException, FileNotFoundException osv.

Når programmet ikke kører, som det skal ifølge sprogets regler, opstår der en Exception, dvs. en undtagelse fra det normale, en fejl!

AT HÅNDTERE FEJL – TRY & CATCH EXCEPTIONS

Man kan håndtere RUN-TIME fejl, altså fejl under programkørslen, via Exceptions, sådan at programmet ikke går ned.
Når programmet ikke kører, som det skal ifølge sprogets regler, opstår der en Exception:
En undtagelse eller afvigelse fra det normale, en fejl!

En 'Exception' bliver kastet, og det afbryder programmet og informerer om en fejl:
Dividerer man fx en værdi med en variabel, som har værdien nul i Java, får man fejlmeldingen: ArithmeticException
Så ved man, at man har lavet den slags basale fejl…

```
try {
        // kør koden, som måske giver run time-fejl, og kaster en Exception
}

catch (exception-type) {
        // fang Exception, hvis koden i try-blokken ikke kører, som den skal
}
```

catch(Exception) fanger alle slags fejl, mens fx catch(FileNotFoundException e) kun fanger den slags…

- ***Trinvis udvikling*** *– hvordan har I forsøgt at bryde problemet ned i mindre del-problemer trinvist?*

- *Brug af* ***pseudokod****e – har I forsøgt at pseudokode mulige løsninger (læg computeren væk)?*

- ***At være bevidst om fejltyper:*** *runtime-fejl, kompilleringsfejl eller logiske fejl*

- ***Debugging*** *via print til konsol eller via breakpoints i editoren ?*

- ***Gummiand-metoden*** *☺(man tænker højt i dialog med en gummiand) ?*

- ***Kommentarer*** *i koden*

- *Planlægning og dokumentation via* ***UML-diagrammer*** *og flowchart*
- *Klassediagrammer, E/R-diagrammer eller use case diagrammer*

- ***Kravspecifikation*** *– funktionelle (hårde) og ikke-funktionelle (bløde) krav*
- *Hvad systemet skal kunne, og hvor godt det skal fungere*

- ***Refaktorering*** *– at strukturere koden - overskuelighed, læsbarhed – til én selv og til andre*

Refaktorering: Eksempler (for at fange ideen)…

- *Sigende variabelnavne, metodenavne & klassenavne?*
- *Store, uoverskuelige kodesegmenter i en metode:*

Kan metoden splittes op i to dele, så den fremstår mere overskuelig?

- *Er der overflødige klasser? (fx en dataklasse uden metoder)*
- *Er der omvendt overfyldte klasser, som har vokset sig uoverskuelige?*

- *Bør et datafelt flyttes til en anden klasse, som i højere grad rummer metoder, der benytter dette datafelt?*
- *Bør et datafelt, som har metoder, der opererer på det, blive flyttet over til et selvstændigt objekt/klasse sammen med disse metoder?*
- *(datafeltet er ikke længere så simpelt, som da man lavede det?)*

- *Bør datafelter gemt i fx en String-array blive flyttet til et selvstændigt objekt/klasse, for at gøre dem mere overskuelige med egne variabelnavne og ikke bare en plads i arrayet, fx info[1], vs. String by = ”Lyngby”*

- *Giver parameterlisten i en metode eller klasse-konstruktør mening, eller er den blevet unødvendig lang?*

Refaktorering: Eksempler…

- *Anvender du flere datafelter fra samme objekt i en metode, hvor det kunne være mere simpelt at give hele objektet med som reference til en metode?*

- *To **subklasser** indeholder samme metode. Læg dem op i baseklassen i stedet.*
- *En metode i en **baseklasse** er kun anvendt i en af subklasserne. Læg metoden ned i subklassen i stedet. Gør det samme med datafelter, som kun er relevant for én subklasse.*
- *To klasser deler datafelter og metoder. Læg disse op i en fælles baseklasse.*
- *En subklasse er næsten identisk med baseklassen. Læg det sammen til kun én klasse.*
- *En subklasse gør kun brug af en smule af dens baseklasse. Gør ikke brug af arv, men opret objekt af den tidligere baseklasse, og kald dens metoder på den måde.*

- *Kompliceret tjek af **betingelser & forgreninger**:*
- *Undgå indlejrede if(…) – flat hierarki så vidt muligt.*
- *Opret evt. metode, som returnerer boolean-værdi, hvor den snørklede betingelse er lagt deri…*
- *Identisk kode ligger i alle forgreninger. Ryk den ud og placér kun ét sted, til sidst.*

Test & debugging: At teste sit program og debugge/fejlfinde undervejs!

Hvorvidt imødekommer programmet de forskellige krav, man har sat op, og hvordan kan vi debugge eller fejlfinde undervejs ?

Dette er en **black-box-test** på systemet –
hvor man ikke kigger ind i koden, men på det kørende programs realisering af krav via håndtering af input og forventet output, altså på systemet som blackbox – men i forhold til krav.
Whitebox-test er omvendt en test af systemets indre, i form af fx kodens gang i løkker, forgreninger mm.

Kravene kan eventuelt sættes op i **funktionelle krav og ikke-funktionelle krav:**
De funktionelle krav er jo den funktionalitet, som systemet skal kunne realisere, og de ikke-funktionelle krav kan tænkes som de *kvaliteter,* hvormed det gøres – i forhold til fx brugervenlighed, designkvaliteter, performance, sikkerhed, opdateringsfrekvens (hvis netværksapplikation), oppetid osv.

Ud over blackbox-test på systemet arbejder man også med at **fejlfinde/debugge** – ved at udskrive **værdier til konsollen** eller via opsatte **breakpoints** i sin editor, fx Processing-editoren eller visual studio code.

En **idiot-test** er, hvor man prøver, om systemet kan håndtere meningsløse input – som fx meget langt eller intet input i en textbox.

Test & debugging: brugertest

Hvorvidt imødekommer programmet de forskellige krav, man har sat op, set fra brugerens perspektiv?

Brugssituation:
Hvor befinder brugeren sig, når programmet anvendes, og har denne kontekst specifikke konsekvenser?

Stil brugeren konkrete & relevante opgaver ud fra kravene til programmet
1: Beskriv opgaven på forståelig måde – 'Find specifik aktiv sag og markér sagen som afsluttet'
2: Opstil succeskriterier eller punkter i observationen, fx –
max. tidsforbrug, antal fejltrin, min. brugertilfredshed samt gennemførselsrate

Tal for succes kan hjælpe med at systematisere resultatet af testens forskellige opgaver.

Suppler med egne observationer og-eller uddybende kvalitativ interview
Hjælp fx brugeren med at sætte ord på deres oplevelse af produktet – ud fra fx tabel med oplevelsesorienterede begreber, https://ddu.systime.dk/?id=248

A/B-test: Giv brugeren to versioner af samme funktion – hvilken er den bedste ud fra opstillede succeskriterier?

Brugertest

Tænke-højt-test:

Få testperson til at tænke højt, mens de bruger produktet

Fx hvordan de åbner en mælkekarton, skifter støvsugerpose, navigerer rundt på en hjemmeside eller løser et specifikt problem med et computerprogram ☺

Giv gerne testpersonen nogle specifikke opgaver. Bland dig ikke, men lyt og notér…

Opgaverne skal være lette at forstå og relevante. Hjælp ikke brugeren undervejs…

Få testpersonen til at uddybe sine tanker i et opfølgende interview via uddybende spørgsmål

Testpersonen skal lære at tænke højt –

fortæl om handlinger, forventninger og vurderinger…

Brugertest på printer – ”papir mangler”, et eksempel:
”Jeg leder efter papirskuffen. Den er ikke nederst på printeren.
Det virker underligt.
Jeg finder papirskuffen midt på forsiden af printeren.
Jeg opdager, der er flere og bliver i tvivl om, hvilken jeg skal vælge.
Jeg opdager, at der står ‘skuffe 2 er tom’ i et lille display. Det kan jeg godt lide osv. osv.

Pseudokode og flowcharts

Pseudokode

- Overskuelig
- ”Disposition”
- Kan være uformel
- Undgår distraktioner under planlægning
- Minder om kode men kan ikke kompilleres
- Kan laves for overordnet hele program eller blot dele med varierende mængde detaljer

Pseudokode

- Eksempel

```
START
    hop1
IF sulten
    THEN spis noget
    ELSE vent en time
        GOTO hop1
END
```

Pseudokode

- Pseudokode:

```
FOR i = 0 til 9
    PRINT i
```

- Faktisk kode i Processing:

```
for (int i = 0; i<10; i++) {
    println(i);
}
```

Pseudokode

- Yatzy eksempel:

```
slag = 3
Gentag:
IF slag > 0
    IF terning ikke låst
        THEN slå terning
    slag–
Slut-gentag
```

Pseudokode

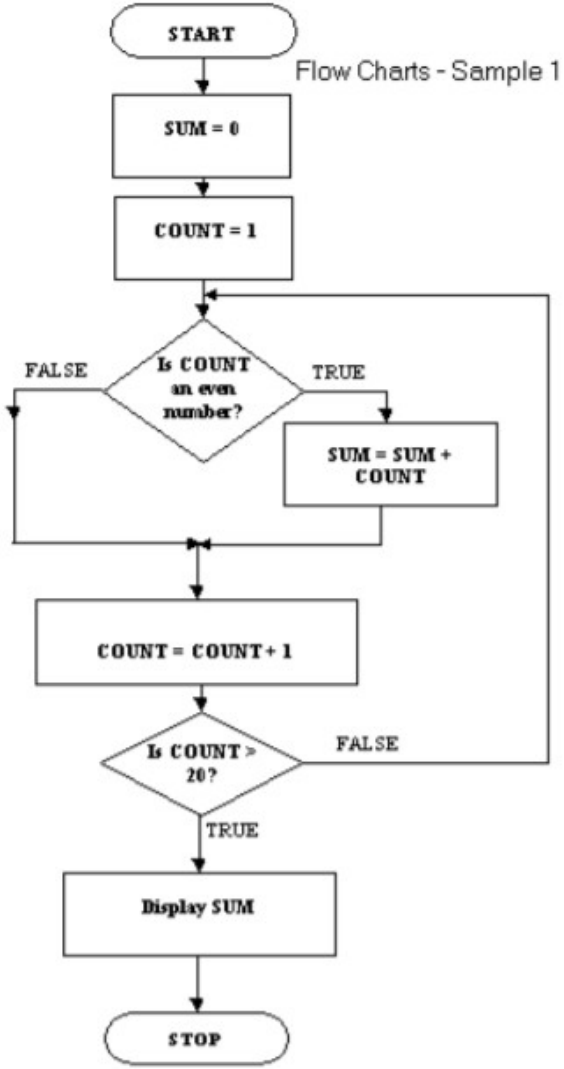
- Retningslinjer
- Ét statement per linje
- Hold fokus på intentionen – ikke implementeringen (hvad i stedet for hvordan)
- Følg standard struktur så vidt muligt
- Brug blokke eller indentering
- Lav gerne flere udgaver f.eks. en overordnet pseudokode til et program og en pseudokode til dele eller funktioner

Flowcharts

- Symboler

Symbol	Navn	Beskrivelse
	Start/slut	Angiver, at algoritmen starter eller slutter.
	Pil	Angiver, i hvilke retningen algoritmen gennemløbes.
	Handling	Angiver, at en handling skal udføres. Bruges til at lave en sekvens.
	Beslutning	Angiver, at en ja/nej beslutning skal tages. Bruges til at lave en forgrening.
	Input/output	Angiver data, som algoritmen skal bearbejde (input) eller har bearbejdet (output).

Flowcharts


START
Flow Charts - Sample 1
SUM = 0
COUNT = 1
FALSE
Is COUNT an even number?
TRUE
SUM = SUM + COUNT
COUNT = COUNT + 1
Is COUNT > 20?
FALSE
TRUE
Display SUM
STOP

Flowcharts

Retningslinjer

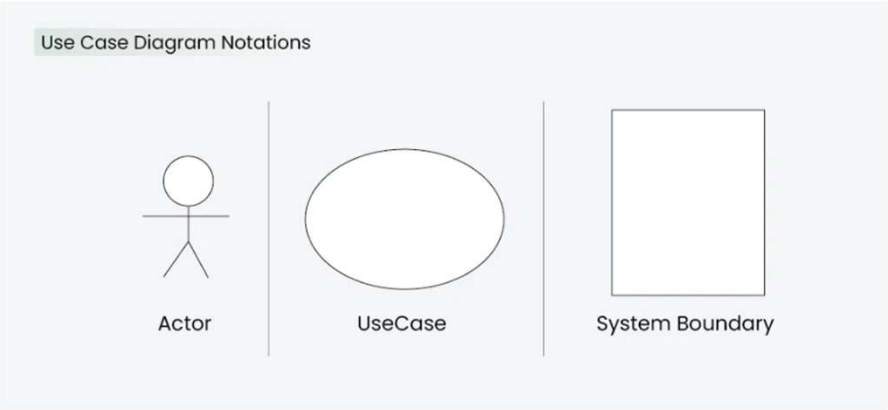
- Ét start og evt. ét slut
- Oppe fra og ned
- Brug forbindelsespile til det hele (intet står alene)
- 1 flowchart per side og helst ikke mere end 1 side
- Titel
- Ca. 15 symboler er max

Brugsmønstre: Use case diagrammer (UML)

https://www.youtube.com/watch?v=zid-MVo7M-E

https://www.lucidchart.com/pages/tutorial/uml-use-case-diagram

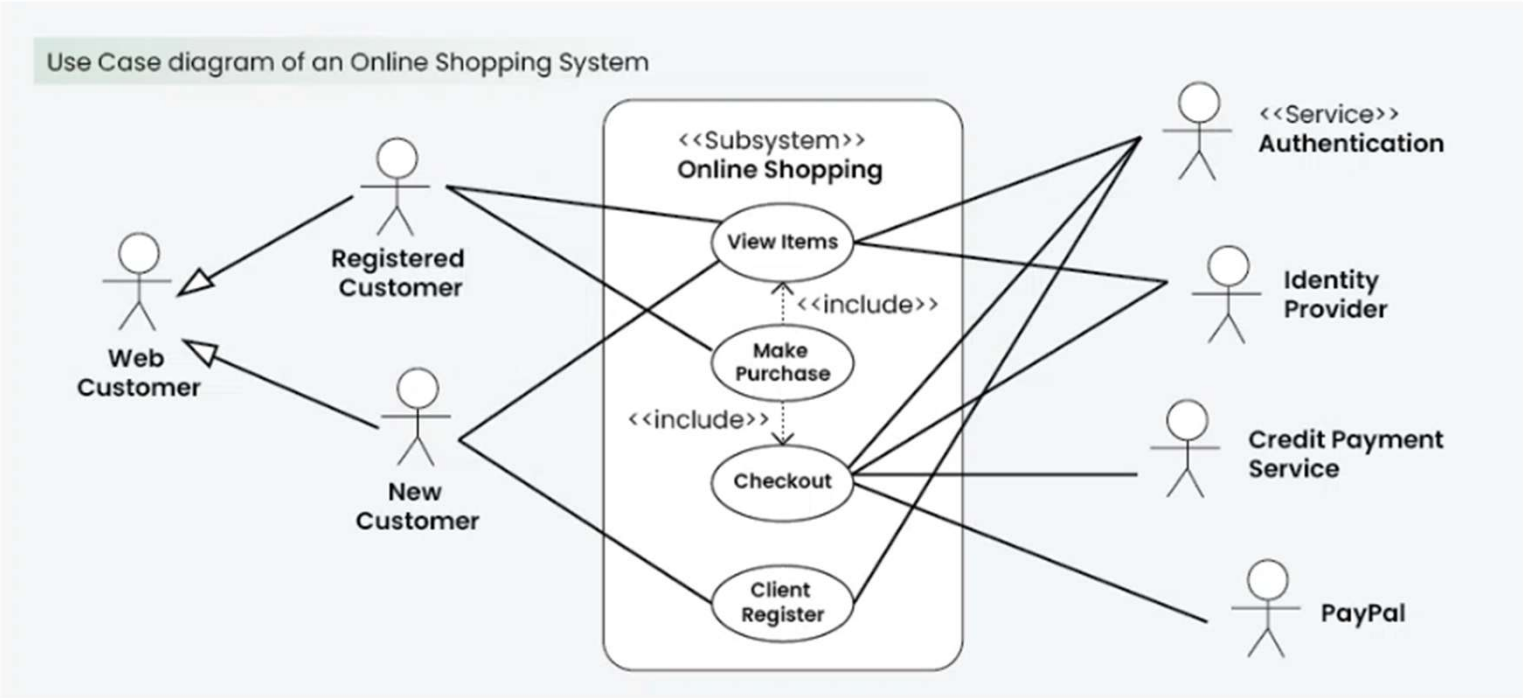
https://www.geeksforgeeks.org/system-design/use-case-diagram/



Aktører – forskellige slags brugere, andre systemer eller hardware-ting som interagerer med systemet.

Use case – ting man gør med systemet – afsluttet handling, mål, "Vis sager", "Opret sag" osv.

Brugsmønstre: Use case diagrammer (UML)



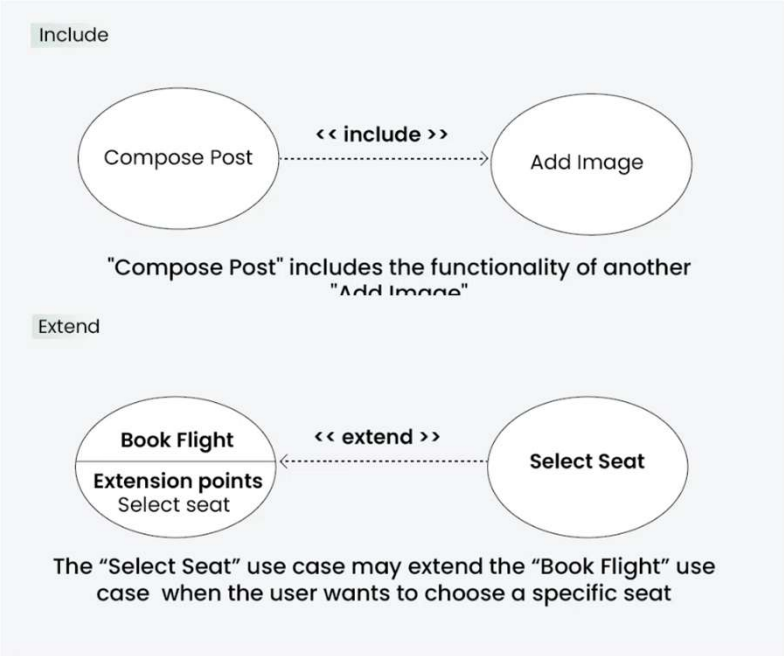
Aktører – forskellige slags brugere, andre systemer eller hardware-ting som interagerer med systemet.

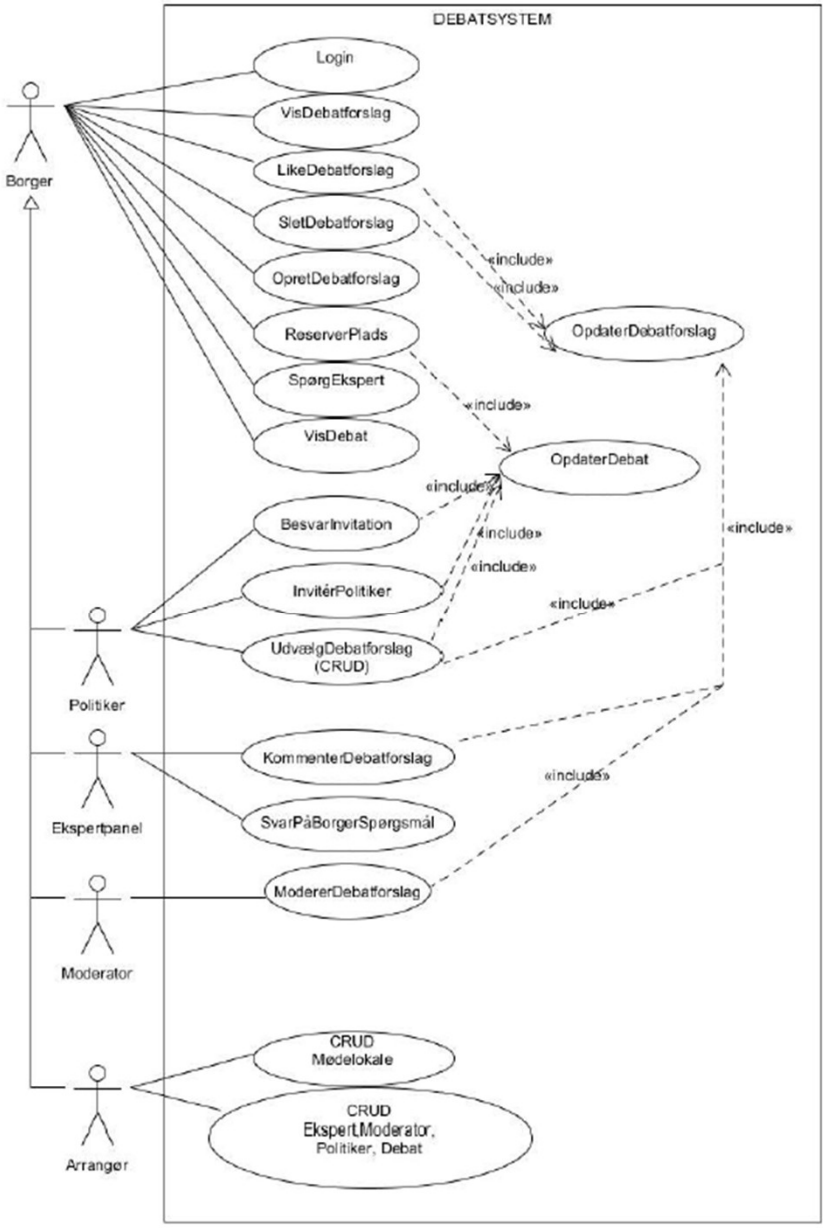
Use case – ting man gør med systemet – afsluttet handling, “Vis sager”, “Opret sag” osv.

Et debatsystem-program:

En borger-aktør kan gøre forskellige ting
En politiker, ekspert, moderator kan det samme som borger
Men kun en moderator kan MordereDebatforslag
(Generalisering (nedarvings-logik))

Include viser at en anden handling er implicit en use case:
De inluderede use cases med stiplede linjer bliver også kørt,
men de genbruges på tværs af andre use cases.
Extend er





Vision & brugsmønstre: kravspecifikation

Projektets vision – overordnet idé og formål kort beskrevet i sammenhængende tekst

Use case diagram: grafisk overblik over projektets brugsmønstre
Use case – er en ting, man gør med systemet – afsluttet handling, mål, "Vis sager", "Opret sag" osv.

Kravspecifikation – funktionelle krav
Opstil brugsmønstre som en oversigt over krav:

Use case navn: Vis antal aktive sager med given art, fx rotter
Formål: Brugeren ønsker info om antal aktive sager med fx rotter
- Brugeren spørger systemet, hvor mange aktive sager med rotter, der aktuelt findes.
- Systemet tæller antal aktive sager og viser resultatet (måden angives ikke her).
- Hvis 0 sager, svarer systemet ikke med antal 0, men med "ingen aktive rottesager" (alternativ rute)

Ikke-funktionelle bløde krav kan angives efterfølgende, fx:
- Resultaterne skal vises grafisk på en for brugeren forståelig og engagerende måde
- Læsning og skrivning af data til tjeneste i skyen må ikke fryse applikationen
- Systemet skal kunne tilbyde download af sagsdata i forskellige filformater, pdf og csv